UNIVERSIDAD ABIERTA PARA ADULTOS (UAPA)

ROSÉA BOUTIQUE

Integración de framework y entrega final

Desarrollo de Aplicaciones Web (ISW-306)
Actividad Práctica 4 · Fase 4 · Semanas 9–10

Proyecto: Tienda virtual Roséa Boutique

Framework: CodeIgniter 4 (PHP) + MySQL/MariaDB

Ambiente local: XAMPP y Composer

Integrantes: [Completar nombres reales]

Facilitador: [Completar]

Fecha: [Completar]

Documento borrador: las pruebas en XAMPP, los aportes y la evidencia del equipo deben completarse antes de entregar.

1. Introducción y objetivos

Roséa Boutique es una aplicación comercial orientada a presentar y vender productos de moda y accesorios de estética coquette. En las primeras fases se construyeron las páginas, la experiencia del usuario y una solución PHP con base de datos MySQL para registro, autenticación, catálogo, compras y gestión administrativa. La Fase 4 busca ordenar esta solución mediante un framework profesional que facilite el mantenimiento y la instalación reproducible.

El propósito es cumplir el resultado RA6: utilizar marcos de trabajo para estandarizar aplicaciones comerciales. De manera concreta, se pretende integrar rutas y controladores, separar modelos y vistas, conservar las operaciones de la Fase 3, preparar cuentas de prueba y documentar la instalación local.

2. Framework elegido y justificación

Se eligió **CodeIgniter 4** porque ya se trabajaba con PHP y MySQL en XAMPP. Migrar a un framework del mismo lenguaje permite reaprovechar la lógica de negocio y el esquema de base de datos sin transformar completamente la solución al entorno de otro lenguaje. Asimismo, CodeIgniter aporta MVC, enrutamiento explícito, modelos de acceso a datos, sesiones, validación de peticiones y servicios del framework.

En la organización propuesta, **app/Config/Routes.php** concentra la correspondencia entre URL, verbo HTTP y controlador; **app/Controllers** recibe solicitudes y valida permisos; **app/Models** interactúa con MySQL; y **app/Views** presenta el contenido HTML. Los recursos estáticos se sirven desde **public/** y los comprobantes privados se conservan en **writable/**.

3. Conceptos del framework aplicados

Los controladores utilizan el objeto de petición/respuesta del framework, las funciones de sesión y el servicio de conexión a base de datos. De esta manera se elimina la dependencia de archivos PHP sueltos con credenciales escritas dentro del código. Las operaciones que modifican datos usan rutas POST con filtro CSRF. La comprobación de rol administrador y de propiedad de cada pedido se realiza antes de mostrar o modificar información privada.

4. Migración desde la Fase 3

El respaldo original incluía scripts independientes como login.php, registro.php, productos.php, checkout.php, crear_pedido.php y archivos del panel administrativo. La migración crea **AuthController** para cuentas y sesiones, **CatalogController** para catálogo y CRUD de productos, **OrderController** para compra y comprobantes, y **AdminController** para usuarios, órdenes y pagos. Los modelos UsuarioModel, ProductoModel y PedidoModel contienen la configuración de las tablas.

Se conservaron las entidades **usuarios, productos, pedidos** y **detalle_pedido**. La compra consulta los precios desde la base de datos, bloquea la fila de inventario mientras crea el pedido y utiliza una transacción para descontar existencias o revertir la operación ante un error. En el panel de administración los productos se pueden crear, editar y ocultar; al ocultarlos se mantiene el historial de referencias previas.

La administración de pedidos comprende actualización de estado, verificación de pagos por transferencia, generación de un código básico de seguimiento y reposición del stock cuando se cancela una orden que aún no estaba cancelada. El sistema impide que un pedido cancelado sea reactivado directamente y limita la consulta de cada pedido a su propietario o a una cuenta administradora.

5. Seguridad y datos de demostración

El archivo .env mantiene fuera del repositorio la configuración de base de datos. La autenticación emplea **password_hash/password_verify** y regenera la sesión al iniciar sesión. Las vistas escapan los datos variables antes de mostrarlos. Los formularios POST incorporan tokens CSRF y los comprobantes de pago se almacenan en la carpeta privada writable/comprobantes, con una ruta que verifica autorización antes de descargarlos.

El ZIP de Fase 3 contenía un respaldo SQL con registros de personas y pedidos. Para la entrega académica se preparó otro script, **database/rosea_demo.sql**, con información ficticia, productos de ejemplo y dos cuentas demostrativas. Así la evaluación puede reproducir la aplicación sin exponer el contenido personal del sistema anterior.

6. Ejecución reproducible

La instalación prevista empieza con XAMPP y PHP 8.2 o superior, Composer 2 y la aplicación base de CodeIgniter 4. El script INSTALAR_XAMPP.ps1 prepara una nueva carpeta dentro de htdocs y copia el código migrado. Se importa rosea_demo.sql mediante phpMyAdmin, se revisa .env y se abre la URL local. El README describe cada etapa e incluye credenciales de prueba ficticias.

7. Pruebas, dificultades y mejoras

Pruebas por realizar en XAMPP: registro, login y cierre de sesión; intento de acceso a una página protegida sin sesión; acceso de cliente a un panel administrativo; alta/edición/ocultamiento de productos; checkout con inventario suficiente e insuficiente; lectura del pedido por su propietario; carga y verificación de comprobante; cancelación con reposición de stock; e instalación limpia en otro equipo.

Resultado: [Completar fechas, responsables, capturas y estado Aprobado/Fallido]. La sintaxis del código migrado fue comprobada con PHP, pero su integración con Apache y MySQL de XAMPP debe probarse en el equipo del grupo.

Dificultades encontradas y cómo se resolvieron: [Documentar problemas reales: configuración de la versión PHP, rutas, conexión de base de datos, permisos, errores y sus soluciones]. No consignar como realizadas acciones que todavía no se han ejecutado.

8. Aportes del equipo

Integrante 1: [Nombre, responsabilidad concreta y enlace/identificador de commit].
Integrante 2: [Nombre, responsabilidad concreta y enlace/identificador de commit].
Otros integrantes: [Completar según el equipo].

9. Conclusión y recomendaciones

La migración a CodeIgniter 4 propone una estructura MVC más clara para las funciones de Roséa Boutique. La separación entre rutas, controladores, modelos y vistas facilita depuración, reutilización y mantenimiento. La base de prueba independiente y la documentación de instalación representan mejoras en reproducibilidad y protección de información.

Antes de cerrar la Fase 4, corresponde verificar las operaciones de principio a fin en XAMPP, corregir cualquier resultado fallido, registrar contribuciones individuales en GitHub, aprobar el Pull Request, publicar la etiqueta v4.0, actualizar ClickUp y presentar el video de 5–8 minutos con al menos dos voces.

Referencias

CodeIgniter Foundation. (2026). *CodeIgniter 4 User Guide*. https://codeigniter.com/user_guide/
Composer. (s. f.). *Introduction to Composer*. https://getcomposer.org/doc/00-intro.md
PHP Documentation Group. (s. f.). *Password hashing*. https://www.php.net/manual/en/book.password.php